

the programmer passes multiple arguments, it assumes these are the values to inspect. These two functions commonly handle lists of values while the program is running, as in the function call `min(allExpenses)`. They also deal with separate arguments the programmer selects while writing the code, such as in `max(0, someNumber)`. Therefore, the functions are designed to accept both kinds of arguments. The following `myMinFunction()`, which is my own implementation of the `min()` function, demonstrates this:

```
def myMinFunction(*args):
    if len(args) == 1:
        ❶ values = args[0]
    else:
        ❷ values = args

    if len(values) == 0:
        ❸ raise ValueError('myMinFunction() args is an empty sequence')

    ❹ for i, value in enumerate(values):
        if i == 0 or value < smallestValue:
            smallestValue = value
    return smallestValue
```

The `myMinFunction()` uses the `*` syntax to accept a varying number of arguments as a tuple. If this tuple contains only one value, we assume it's a sequence of values to inspect ❶. Otherwise, we assume that `args` is a tuple of values to inspect ❷. Either way, the `values` variable will contain a sequence of values for the rest of the code to inspect. Like the actual `min()` function, we raise `ValueError` if the caller didn't pass any arguments or passed an empty sequence ❸. The rest of the code loops through values and returns the smallest value found ❹. To keep this example simple, `myMinFunction()` accepts only sequences like lists or tuples rather than any iterable value.

You might wonder why we don't always write functions to accept both ways of passing a varying number of arguments. The answer is that it's best to keep your functions as simple as possible. Unless both ways of calling the function are common, choose one over the other. If a function usually deals with a data structure created while the program is running, it's better to have it accept a single parameter. If a function usually deals with